

Write an HTTP server from the ground up in 9 minutes with Python

9 min read · Jan 30, 2023



Stephen Biston

Follow



Listen



Share

“If you wish to make an apple pie from scratch, you must first invent the universe” — Carl Sagan

Why?

If you're reading this, I probably don't need to convince you that building your own HTTP server from scratch is a worthwhile pursuit. In fact, it isn't. Never say never... but you should *never* use a homegrown HTTP server in a production environment. Hell, you should probably never use one even in a *test* environment. That said, if you're like me and just like to know how things work at a more basic level, there's a certain satisfaction about taking things down to the fundamentals of web technology and dispelling any of the “magic” that surrounds it.

How?

Sockets. It's sockets all the way down. Sockets are the basic building blocks of connections between computers. Fundamentally, a socket is a “file descriptor”, a reference to an input/output stream, from which a program read and to which it can write.

While not quite technically accurate, I like to think of a socket as a file that two programs can use simultaneously, similar to colleagues working on the same Google Doc concurrently. Content written from one computer can be consumed and read

by the other computer and vice versa. A problem that is encountered here is that computers fundamentally only know binary values — 1s and 0s — so how can they know what “kind” of data is being communicated from an entirely separate computer? How can it know if these bytes represent text, images, numbers, or something else entirely? The solution is for a standard data structure pattern to be known independently by each computer, allowing each to match the incoming data to a known pattern and by doing so determine the intended meaning. These patterns are known as *protocols*.

When each computer agrees on a particular structure for the data they write and read, an effective stream of meaningful communication can be established. For our work here, the two protocols we are concerned with are TCP (TCP/IP) and, you guessed it, HTTP.

TCP Sidebar

TCP isn't our focus here, so we won't go too deep into the details. Suffice it to say it's a protocol that defines a method for initiating and maintaining a basic connection between two computers and transferring data in a request-response pattern. Separate protocols like HTTP can use TCP connections to implement their own more specific request-response pattern: HTTP specifies the pattern and structure of the messages to be transferred, whereas TCP is responsible for initializing the connection and the actual process of transferring the data.

For what it's worth, the underlying TCP details are low-level enough to be abstracted even in the corresponding C code, delegated to OS system calls.

```
#include <sys/socket.h>

// Create server socket file descriptor, SOCK_STREAM indicating a TCP socket
server_fd = socket(AF_INET, SOCK_STREAM, 0);
```

Getting Started with Sockets

In Python, complicated things get a lot easier, but it can gloss over a lot of the depth which you might get by using a language like C. That being said, this is a 9-minute server, not a 2-day server so we're going to use Python anyway and I'll add some of the more pertinent details along the way.

For our purposes, a socket server is simply a server that listens for incoming data on a specific socket which is identified by an address and port.

server.py (Python 3.10+)

```
import socket
import select

if __name__ == "__main__":
    # Get socket file descriptor as a TCP socket using the IPv4 address family
    listener_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
    # Set some modes on the socket, not required but it's nice for our uses
    listener_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

    address_port = ("127.0.0.1", 8080)
    # Reserve address and port
    listener_socket.bind(address_port)
    # Listen for connections, a maximum of 1
    listener_socket.listen(1)
    print("Server listening @ 127.0.0.1:8080")

    # Loop indefinitely to continuously check for new connections
    while True:
        # Poll the socket to see if there are any newly written data, note exception
        read_ready_sockets, _, _ = select.select(
            [listener_socket], # list of items we want to check for read-readiness
            [], # list of items we want to check for write-readiness (not implemented)
            [], # list of items we want to check for "exceptional" conditions
            0 # timeout of 0 seconds, makes the method call non-blocking
        )
        # If a value was returned here then we have a connection to read from
        if read_ready_sockets:
            # select.select() returns a list of readable objects, so we'll iterate
            for ready_socket in read_ready_sockets:
                # Accept the connection from the client and get its socket object
                client_socket, client_address = ready_socket.accept()

                # Read up to 4096 bytes of data from the client socket
                client_msg = client_socket.recv(4096)
                print(f"Client said: {client_msg.decode('utf-8')}")
```

```
# Send a response to the client, notice it is a byte string
client_socket.sendall(b"Welcome to the server!\n")
try:
    # close the connection
    client_socket.close()
except OSError:
    # client disconnected first, nothing to do
    pass
```

Read over the comments in the above file for a step-by-step description of the process.

The above file is a basic Python script that sets up a socket that listens for incoming connections at the local address 127.0.0.1 using port 8080. Using the Python `select` module's `select()` function we're able to poll our socket to see if there are any data for us to read from the socket and if we do we accept the incoming connection, `print()` the message that the client sent and we send back a welcome message. Lastly, we close the client connection and continue our loop so we can pick up the next connection.

We can prove the functionality of this server easily by starting the server and sending a TCP message to it with a client, for instance using the `nc` (netcat) Linux tool.

Start server

Copy the `server.py` file from above into a new Python file, then in a terminal window navigate to the folder where that file is saved.

```
$ python server.py
Server listening @ 127.0.0.1:8080
```

Send TCP message to the server

The terminal window where you initiated the server will be blocked as the server runs in an infinite loop, so open a separate terminal window (doesn't matter what the working directory for this terminal is).

```
$ echo "Hello I am the client!" | nc 127.0.0.1 8080
```

```
Welcome to the server!
```

After sending the message, in addition to the response to the client seen above you should also see the server print a “Client said:...” message to indicate the client’s message was received. And that’s all there is to it! That’s a functional TCP server. A little additional work could be done to maintain the open client connection instead of closing immediately, but for our purposes this is sufficient.

What about HTTP?

Now we’ve got a minimally viable TCP server that can respond to requests, but this article is about building an HTTP server, not a TCP server.

Get Stephen Biston’s stories in your inbox

Join Medium for free to get updates from this writer.

Enter your email

Subscribe

With your new TCP server running via `python server.py` open a tab in your browser at `http://127.0.0.1:8080`. At this point you don’t get a nice webpage served back to you, rather, your browser gives you an error about an “Invalid Response” or something similar. But check the terminal window where your server is running, you should notice a pretty large page of output, the first few lines of which should be roughly similar to the below:

```
Client said: GET / HTTP/1.1
Host: 127.0.0.1:8080
Connection: keep-alive
Cache-Control: max-age=0
sec-ch-ua: "Not_A Brand";v="99", "Google Chrome";v="109", "Chromium";v="109"
sec-ch-ua-mobile: ?0
sec-ch-ua-platform: "macOS"
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36
```

```
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/
...
```

What you're seeing here is the data that makes up the HTTP request that your browser sends to a server in hopes of receiving an HTTP response containing webpages and other such content. While not strictly tied to TCP, HTTP servers are underpinned with a TCP server that provides the transport layer between the client and the server that allows reliable transmission of HTTP data. As a TCP server, our server can receive and read the data sent by the browser just fine. The only problem is that the browser expects an HTTP response back and it's not getting it, hence the "Invalid Response" error.

The solution at this point should be clear: we need to send the browser a valid HTTP response. How do we know what that looks like?

Who Decides?

Recall earlier when we discussed the problem of two separate computers communicating with each other over sockets: when all data has the same underlying composition (binary), how can the computers know whether that binary represents text/images/video/etc? The answer is to have shared, independently understood data structures that the client and server are programmed to adhere to called *protocols*.

As it turns out, HTTP stands for **H**ypertext **T**ransfer **P**rotocol. It is a specification for the structure and syntax of text requests and responses between a server and client that allows both parties to understand what is being requested and received.

With that understanding, it should be clear that for our TCP server to "become" an HTTP server, we simply need to respond to our browser's valid HTTP request with a valid HTTP response. The definition of a valid HTTP message can be found in [RFC 911](#) provided by the Internet Engineering Task Force (IETF). It's a bit of a dry read, but you can review that to get a solid understanding of what makes a valid HTTP response and use it directly as a specification for your server.

However, in the interest of the 9-minute server, I've already done that and wrote a little more code to make things adequately conformant to the protocol:

server.py (Python 3.10)

```
# replacing the client_socket.sendall call from before
client_socket.sendall(
    bytes(f"""HTTP/1.1 200 OK\r\nContent-type: text/html\r\nSet-Cookie: ServerN
\r\n
<!doctype html>
<html>
    <head/>
    <body>
        <h1>Welcome to the server!</h1>
        <h2>Server address: {address_port[0]}:{address_port[1]}</h2>
        <h3>You're connected through address: {client_address[0]}:{client_a
        <body>
            <pre>{client_msg.decode("utf-8")}</pre>
        </body>
    </body>
</html>
\r\n\r\n
""", "utf-8")
)
```

The above changes just swap out the response we're sending back to the client; as anticipated, everything else can stay the same. Reviewing the updates you'll notice a status line at the top `HTTP/1.1 200 OK` which should look familiar to most followed by a few linefeed (`\r\n`) characters, the HTTP headers for `Content-type` and `Set-Cookie`, then lastly an HTML message body. Note that headers are not required but are added for demonstration purposes.

Run this updated `server.py` file and open your browser to `http://127.0.0.1:8080`, you should see something like this:



No more browser complaints, we have a valid HTTP response!

Voila! Our very own web server! It's accepting HTTP requests and emits HTTP responses good enough for a real web browser.

With a little imagination, you can conceive of how one might be able to dress this up with the common popular web server features like middleware that routes requests based on path or the ability to load the HTML message bodies from files elsewhere in the project.

Final Thoughts

For those who didn't follow along, see the final file as a gist [here](#).

While the server created here is wildly far from a true production-grade web server (if you have a free afternoon feel free to read over the [nginx source code](#) for a better example), hopefully, you finish this post with more clarity on the fundamental nature of the software. Primarily:

1. Sockets, which are kind of like files that multiple programs can have open at once, power the ability of two separate computers to communicate with each other.
2. TCP is a commonly used protocol, implemented at an OS level, which defines the pattern for request/response-style communications.
3. In our case, we use TCP for transporting messages, but HTTP simply defines a specific syntax and structure for requests and responses between a client and server; if you adhere to the functionality described by the HTTP RFC specification you will effectively have an HTTP server.

HTTP is one protocol, but you can build off of the basic socket server concept more and more to implement protocols other than HTTP, such as [WebSocket](#) or [IRC](#). Give it a try!

P.S.

Let me know if you know of a good reason to use a self-made HTTP server for anything other than demonstration/educational purposes!

See Also

- My over-engineered, under-featured Socket/HTTP/WebSocket server project that led to this post: <https://github.com/sjb9774/stevesockets>
- More HTTP-related RFCs: https://developer.mozilla.org/en-US/docs/Web/HTTP/Resources_and_specifications
- The ready-made socket server implementation provided by Python out-of-the-box: <https://docs.python.org/3/library/socketserver.html>

Python

Web Development

Servers

Web

Sockets



Follow

Written by Stephen Biston

99 followers · 3 following

Web Dev & Technical Lead with over a decade of experience. Did my content help you out? Feel free to buy me a coffee: <https://ko-fi.com/stephenbiston>

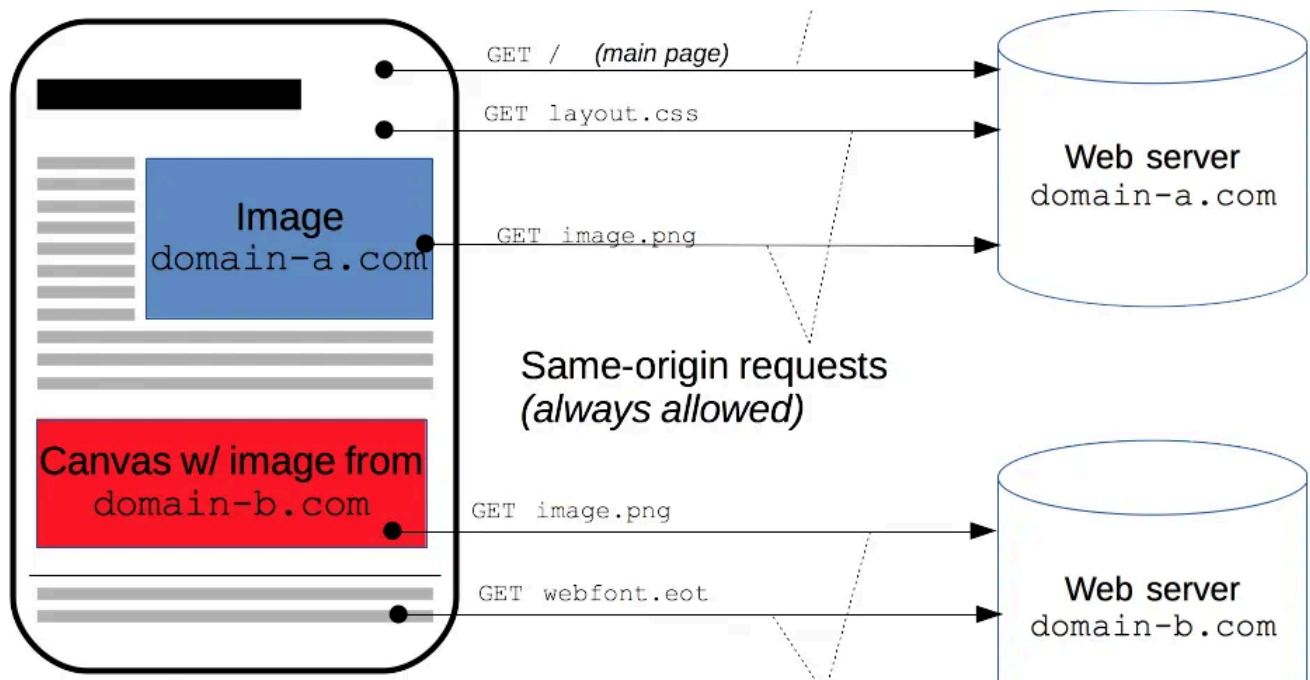
No responses yet



Write a response

What are your thoughts?

More from Stephen Biston



 Stephen Biston

Resolve CORS Errors Once and For All: Three Methods

The Three Approaches for Resolving CORS Errors

Open in app ↗

[Sign up](#)

[Sign in](#)

Medium

 Search



Use your EventBridge credentials from your Amazon Web Services account to set up an event source. [Learn more about integrating Amazon EventBridge.](#)

AWS account ID

123412341234

Log in to the AWS dashboard to get your account ID

AWS region

US West (N. California)

Source name

shopify-webhooks-event-source

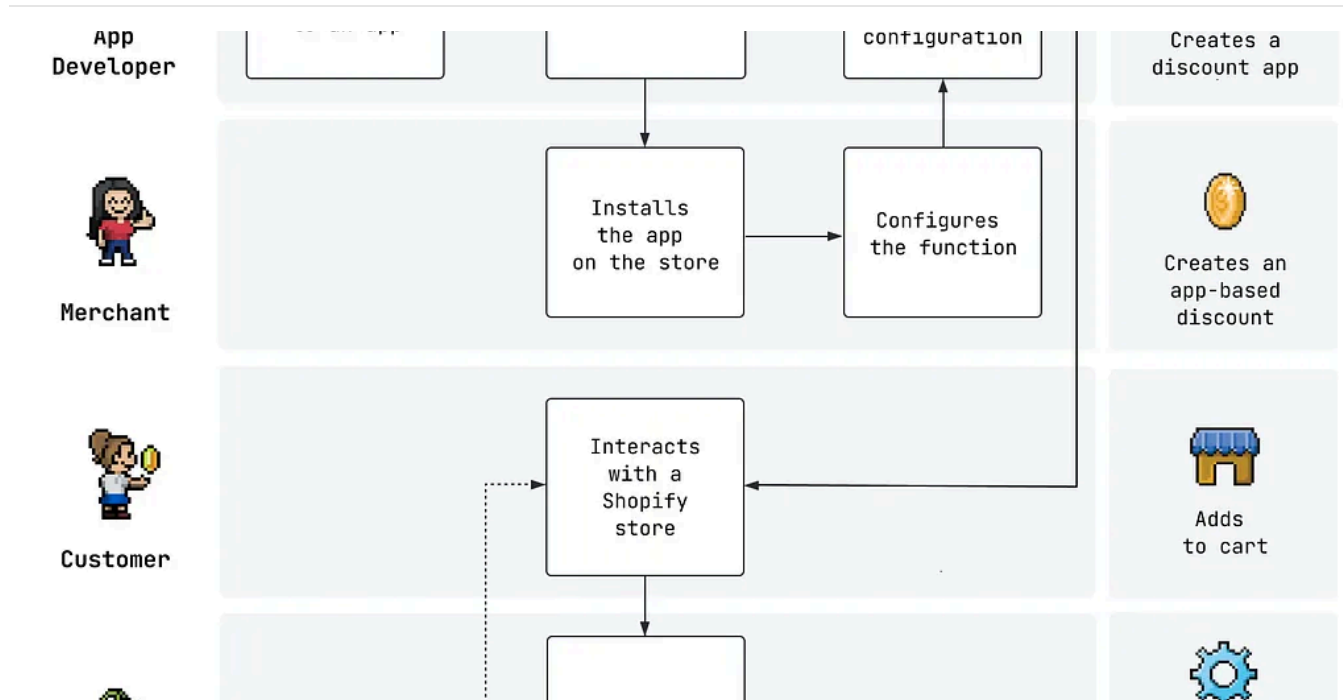


Stephen Biston

Shopify: AWS Serverless Webhooks

Robust Webhooks Integrations using AWS EventBridge and SQS

Mar 6, 2023 🖱 26



Stephen Biston

Getting Started with Shopify Functions

Advanced Shopify: Customizing the Backend

🌟 Feb 13, 2023 🖱 36 💬 3



```
> Static
> Global
> Registers

WATCH

CALL STACK
> 1: tid=3619069 "main" PAUSED
v 2: tid=3619088 "test..." PAUSED ON BREAKPOINT
rust_function_demo_ext::function main.rs
core::ops::function::Fn::call @core::o...
shopify_function::run_function_with_input

42 .....discount_application_strategy: output::DiscountApplicationStrategy::FIRST,
43 .....};
44 .....
45 .....// If there is a metafield value defined, parse out our config otherwise immediately return "no discount" response
46 .....let config = match input.discount_node.metafield {
47 .....    Some(input::InputDiscountNodeMetafield { value }) =>
48 .....        Configuration::from_str(&value),
49 .....    None => return Ok(no_discount),
50 .....};
51 .....
52 .....let config_discount_type = &config.discount.discount_type;
53 .....let config_discount_amount = &config.discount.value;
54 .....let p_string = "percentage".to_string();
55 .....// confirm we can match the discount type to "percentage" -- notably nothing happens if the discount type is anything
56 .....// assuming we have a "percentage" discount then output_discount is set as a Percentage object with a discount value
57 .....let output_discount = match config_discount_type {
58 .....    p_string => output::Percentage { value: config_discount_amount.to_string() }
59 .....};
60 .....
61 .....
62 .....let first_merch = &input.cart.lines[0].merchandise;
63 .....// if there's at least one item in the cart (should always be) and if so track the id of that first item
64 .....let first_id = match first_merch {
65 .....    input::InputCartLinesMerchandise::ProductVariant(variant) => &variant.id,
66 .....    _ => ""
67 .....};
68 .....
69 .....// Return a response applying our configured discount amount to the first line item in the cart with no quantity limit
70 .....return Ok(output::FunctionResult {
71 .....    discounts: vec![output::Discount {
72 .....        message: Some("Your Shopify Function is working!".to_string()),
```



Stephen Biston

Debugging Shopify Functions

How to use the Function Runner and Debug WASM and Rust

Feb 22, 2023



132



3



See all from Stephen Biston

Recommended from Medium

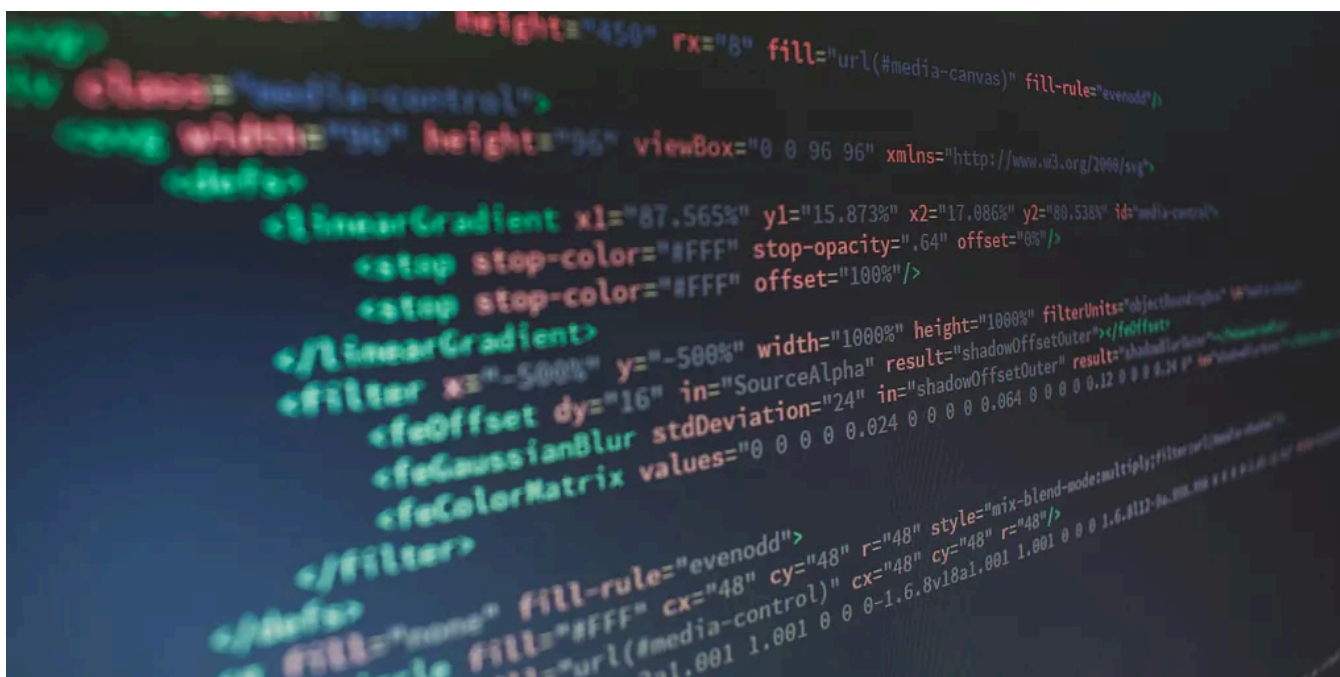


PY In Python in Plain English by Suleman Safdar

How I Built 6 Micro-Tools in Python That Earn Me Passive Income Daily

I stopped chasing big projects and started building tiny, high-impact Python scripts. These libraries helped me automate, scale, and...

★ Aug 7 🖱 1.1K 💬 16

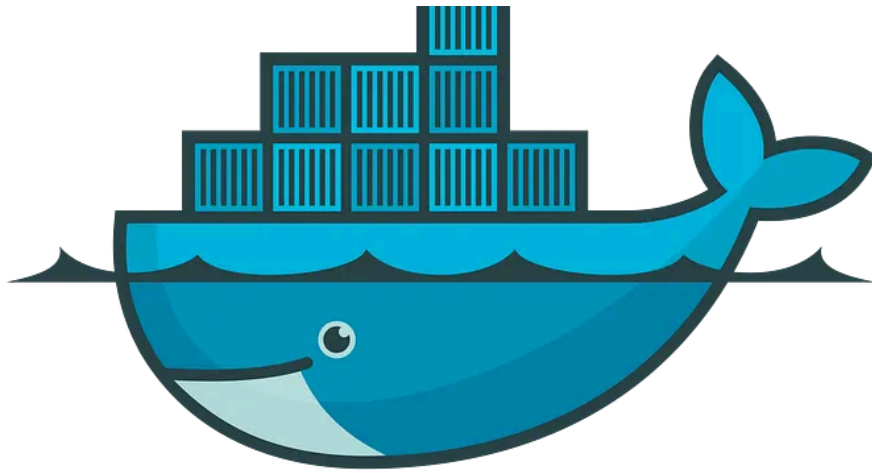


Python In Top Python Libraries by Mariyam Mahmood

How I Took a Local Script and Made It Run Anywhere with Docker

Containerizing My Python Data Pipeline

★ 5d ago 🖱 51 💬 1



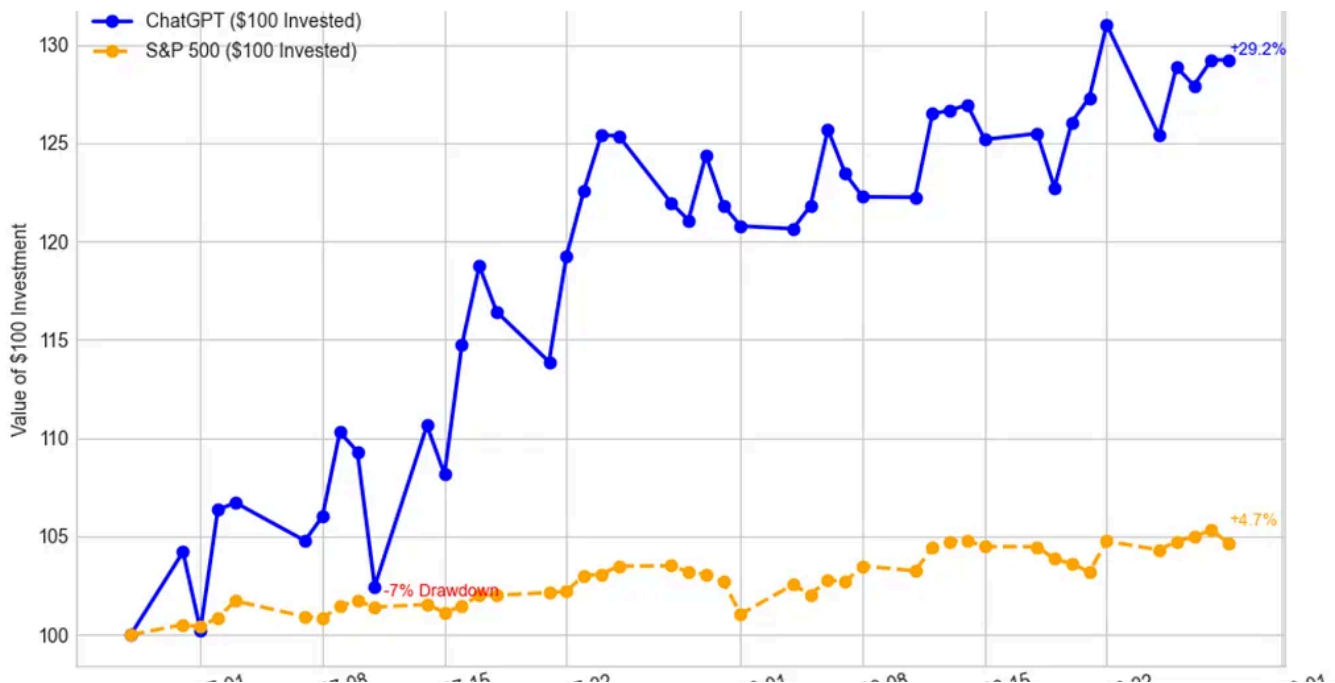
docker

 Abhinav

Docker Is Dead—And It's About Time

Docker changed the game when it launched in 2013, making containers accessible and turning “Dockerize it” into a developer catchphrase.

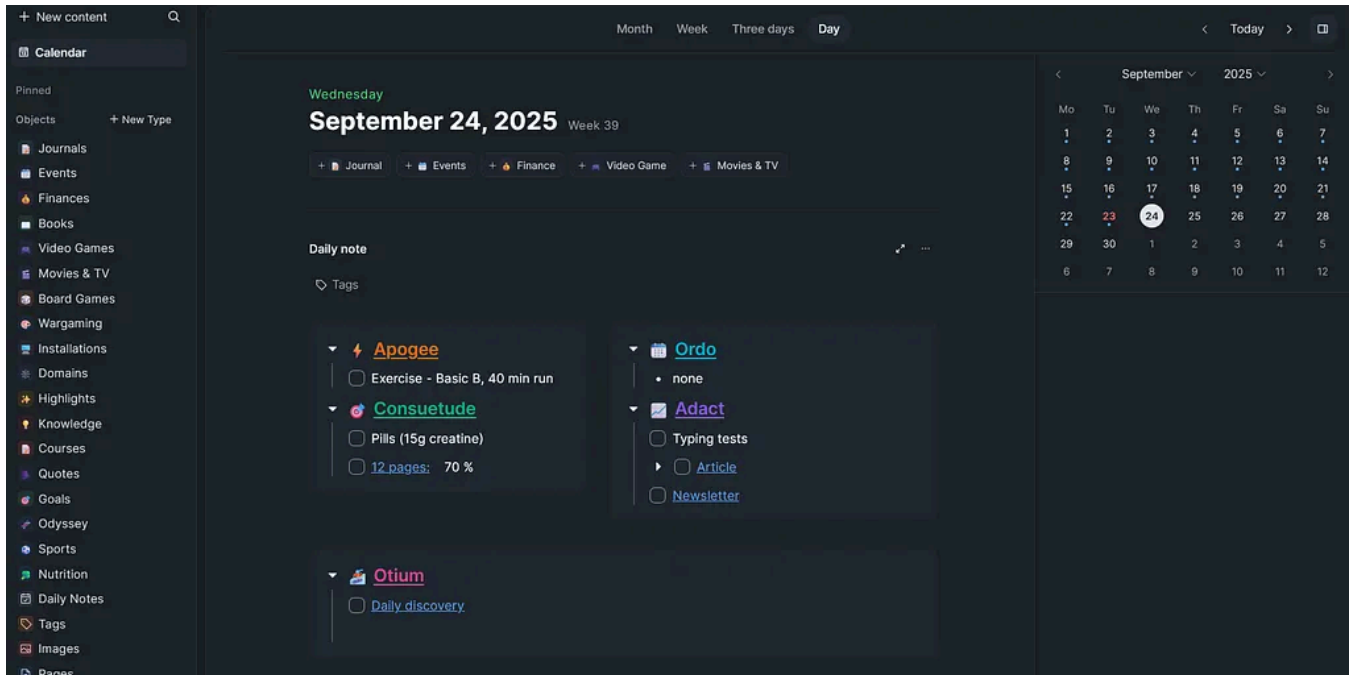
★ Jun 9 🖱 6.7K 💬 187

 In Coding Nexus by Civil Learning

I Handed ChatGPT \$100 to Trade Stocks—Here's What Happened in 2 Months.

What happens when you let a chatbot play Wall Street? It's up 29% while the S&P 500 lags at 4%.

🌟 Sep 3 🖱️ 4.4K 💬 112

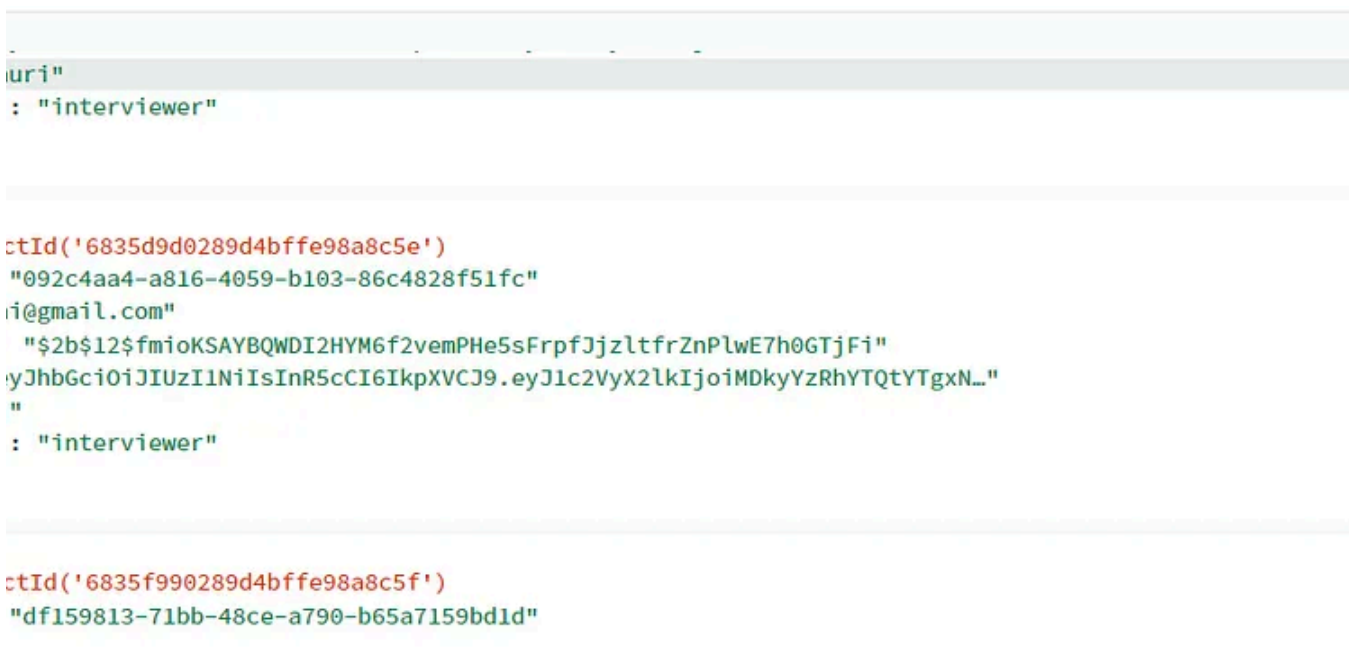


🇹🇹 Tosny

7 Websites I Visit Every Day in 2025

If there is one thing I am addicted to, besides coffee, it is the internet.

🌟 Sep 23 🖱️ 2K 💬 66





Gaurikhard

Creating the Login and Registration System with FastAPI & React

In our previous post, we set up the React frontend and FastAPI backend, laid the groundwork with Docker and Swagger, and ensured our app...

May 27  1



See more recommendations